

S26-063-D-CAVE-OT

Context-Aware Vulnerability Engine

Project Team

Amna Asif	22I-8777
Abdul Mueed Malik	22I-1622
M.Nameer Khan	22I-1695

Session 2022-2026

Supervised by

Dr, Subhanullah



Department of Cyber Security

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

May, 2026

Contents

1	Introduction	1
1.1	Existing Solutions	1
1.2	Problem Statement	2
1.3	Scope	3
1.4	Modules	3
1.4.1	Module 1: Asset Discovery and Contextual Risk Scoring	4
1.4.2	Module 2: Attack Path Analysis and Remediation Advisor	4
1.5	Work Division	4
2	Project Requirements	7
2.1	Event-Response Table	7
2.2	Functional Requirements	12
2.2.1	Module 1: Asset Discovery and Inventory Management	12
2.2.2	Module 2: Context-Aware Risk Scoring Engine	12
2.2.3	Module 3: AI Policy and Compliance Gatekeeper	13
2.2.4	Module 4: Attack Path Analysis and Correlation	13
2.2.5	Module 5: Web Dashboard and Reporting Interface	14
2.3	Non-Functional Requirements	14
2.3.1	Reliability	14
2.3.2	Usability	14
2.3.3	Performance	15
2.3.4	Security	15
3	System Overview	17
3.1	Architectural Design	17
3.1.1	Box and Line Diagram	18
3.1.2	Finalized Architecture: Layered (N-Tier) Pattern	20
3.2	Data Design	23
3.2.1	Overview	23
3.2.2	Data Entities and Storage	23
3.2.2.1	Assets	23
3.2.2.2	Vulnerabilities	25
3.2.2.3	Alerts	26

3.2.3	Entity Relationship	27
3.2.4	Data Processing and Transformation	28
3.2.5	Indexing Strategy	28
3.2.6	Intermediate Data Structures	29
3.3	Domain Model	29
3.4	Design Models	32
3.4.1	Activity Diagram	32
3.4.2	Class Diagram	35
3.4.3	Class-Level Sequence Diagram	38
3.4.4	State Transition Diagram	41
4	Implementation and Testing	45
4.1	Algorithm Design	46
4.2	External APIs/SDKs	48
4.3	Testing Details	50
4.3.1	Unit Testing	50
5	Conclusions and Future Work	55
5.1	Conclusion	55
5.2	Future Work	56
	References	58

List of Figures

3.1	High-Level Box and Line Diagram of CAVE-OT	19
3.2	Finalized 4-Layer Software Architecture Diagram of CAVE-OT	22
3.3	Entity Relationship Structure of the CAVE-OT Database	28
3.4	Domain Model for CAVE-OT	31
3.5	Activity Diagram — CAVE-OT Pipeline End-to-End Flow	33
3.6	Class Diagram — CAVE-OT System Components and Relationships . . .	35
3.7	System-Level Sequence Diagram — CAVE-OT End-to-End Interaction Flow	38
3.8	State Transition Diagram — Asset Entity Lifecycle in CAVE-OT	41
4.1	Algorithm 1: TF-IDF Cosine Similarity CVE Mapping	46
4.2	Algorithm 3: UPSERT-Based Database Persistence	48

List of Tables

1.1	Comparison of Existing OT Security Solutions	2
1.2	Work Division Among Team Members	4
2.1	Event-Response Table for CAVE-OT	7
3.1	Module-to-Layer Mapping (4-Layer Software Architecture)	21
3.2	Assets Table Schema	24
3.3	Vulnerabilities Table Schema	25
3.4	Alerts Table Schema	27
4.1	External APIs and SDKs used in CAVE-OT	49
4.2	Unit Test Cases for CAVE-OT System	53

Chapter 1

Introduction

Industrial IT/OT environments require high availability, safety, and continuous operation, making traditional IT-centric vulnerability management approaches insufficient. Existing solutions primarily rely on static CVSS scores and active scanning techniques, which often ignore operational impact and may disrupt sensitive OT systems such as SCADA and PLCs.

The proposed project introduces an **AI-driven, context-aware vulnerability management system (CAVE-OT)** that prioritizes vulnerabilities based on exploitability, asset criticality, and industrial constraints. Instead of focusing only on detection, the system emphasizes **decision support, explainability, and operational safety**, enabling security teams to make informed and OT-safe remediation decisions.

1.1 Existing Solutions

Several vulnerability management solutions are currently used in industrial environments, such as **Nozomi Networks** and **Tenable.ot**. These platforms provide visibility into OT assets and vulnerabilities; however, they still follow traditional approaches that are not fully aligned with industrial requirements.

Most existing systems rely heavily on CVSS-based prioritization, which assigns severity scores without considering real-world exploitability, production impact, or asset criticality. Additionally, these tools lack attack-path reasoning, making it difficult to understand how multiple vulnerabilities can be chained to compromise critical systems.

Another limitation is the reliance on active scanning techniques, which can disrupt sensitive OT devices and affect system availability. Furthermore, these solutions provide limited decision support and lack explainability, forcing security teams to manually analyze large volumes of alerts.

Table 1.1: Comparison of Existing OT Security Solutions

System Name	System Overview	System Limitations
Nozomi Networks	Provides OT asset visibility and network monitoring with strong industrial awareness.	Lacks attack-path analysis, limited decision explanation, and no context-aware prioritization.
Tenable.ot	Identifies vulnerabilities in OT environments and integrates with enterprise security tools.	Limited OT context awareness, no attack-path reasoning, and weak support for operational impact analysis.
Traditional CVSS-Based Tools	Use standardized scoring (CVSS) to rank vulnerabilities based on severity.	Ignore exploit likelihood, asset criticality, and production impact, leading to poor prioritization.

1.2 Problem Statement

The increasing convergence of IT and OT systems in industrial environments has significantly expanded the attack surface, making these systems more vulnerable to cyber threats. Traditional vulnerability management solutions are designed for enterprise IT systems and rely heavily on static CVSS scores, which fail to reflect real-world risk in industrial settings.

Research shows that only a small percentage of vulnerabilities are actually exploited, yet security teams are overwhelmed with large volumes of alerts. This creates confusion in prioritizing which vulnerabilities should be addressed first. Moreover, vulnerabilities with the same CVSS score can have very different impacts depending on their location in the system. For example, a vulnerability in a SCADA system may be far more critical than one in a non-essential IT service, even if both share the same severity score.

Existing tools also lack the ability to analyze multi-step attack paths across IT and OT networks, limiting situational awareness. Additionally, active scanning methods pose risks of disrupting critical operations, which is unacceptable in industrial environments where availability and safety are top priorities.

As a result, security teams struggle to make informed decisions without risking downtime, safety, or financial loss. This highlights the need for a context-aware, explainable, and OT-safe vulnerability management system that prioritizes risks based on real-world impact rather than static severity scores.

1.3 Scope

The scope of this project is to design and develop an AI-driven, context-aware vulnerability management and attack path analysis system specifically for industrial IT/OT environments. The system focuses on improving decision-making by providing prioritized, explainable, and safe remediation recommendations.

The project includes passive asset discovery using audit logs and network traffic analysis to maintain an accurate inventory of IT and OT assets without disrupting operations. It also includes a context-aware risk scoring engine that evaluates vulnerabilities using CVSS, EPSS, asset criticality, and operational impact.

Attack path analysis is included to identify how multiple vulnerabilities can be chained across IT and OT networks to reach critical systems. The system also incorporates an LLM-based remediation advisor that provides explainable and policy-compliant recommendations with human-in-the-loop decision-making.

In-Scope:

- Industrial IT/OT networks (SCADA, PLCs, IT servers)
- Passive monitoring and asset discovery
- Context-aware vulnerability prioritization
- Attack path analysis across IT/OT environments
- Human-in-the-loop remediation recommendations

Out-of-Scope:

- Active vulnerability scanning or exploitation
- Zero-day vulnerability detection
- Deployment in live industrial environments
- Automatic patching or remediation execution
- Testing on real industrial hardware

1.4 Modules

The system is divided into two main modules, each responsible for core functionality in vulnerability management and attack path analysis.

1.4.1 Module 1: Asset Discovery and Contextual Risk Scoring

This module focuses on identifying IT/OT assets and evaluating vulnerabilities in a non-intrusive manner using contextual information.

1. Hybrid asset discovery using audit logs and passive network monitoring.
2. Context-aware risk scoring based on CVSS, EPSS, asset criticality, and operational impact.

1.4.2 Module 2: Attack Path Analysis and Remediation Advisor

This module analyzes how vulnerabilities can be exploited in sequence and provides explainable remediation guidance.

1. Attack path modeling and visualization across IT and OT networks.
2. LLM-based remediation advisor with human-in-the-loop decision support.

1.5 Work Division

The project work is divided among team members based on their expertise to ensure efficient development, integration, and testing of all system components. Some tasks such as system design, evaluation, and documentation are performed collaboratively.

Table 1.2: Work Division Among Team Members

Name	Registration No.	Responsibilities
Amna Asif	22I-8777	• Hybrid asset discovery implementation • PostgreSQL database design and integration • Dashboard for asset visualization • LLM + RAG integration • Human-in-the-loop decision interface

Abdul Mueed Malik	22I-1622	• Context-aware risk scoring engine • Machine learning models (Random Forest/XGBoost) • Data preprocessing and KEV dataset integration • Attack path modeling using NetworkX • Visualization for SOC dashboard
M. Nameer Khan	22I-1695	• Feature engineering and asset criticality modeling • OT-specific CIA weighting • Model validation and testing • OT-safe remediation strategies • Policy compliance in recommendations • Dashboard reporting support

Chapter 2

Project Requirements

This section outlines the requirements necessary for the successful development of the proposed system, CAVE-OT. The requirements are categorized into functional and non-functional requirements. Functional requirements describe the core operations and features of the system, while non-functional requirements define performance, usability, security, and system constraints.

2.1 Event-Response Table

Table 2.1 captures every significant event in CAVE-OT, the source that generates it, the input data involved, and the system response it triggers. The table covers the full processing pipeline from passive asset discovery through risk scoring, attack path analysis, policy validation, LLM-based remediation, and operator decision handling.

Table 2.1: Event-Response Table for CAVE-OT

#	Event	Source	Input Data	System Response
1	New network packet detected on monitored interface	PassiveMonitor (Scapy / PCAP)	Raw network packet	Parse packet; extract IP address, hostname, protocol, and asset type (IT or OT); add new asset to AssetInventory and persist to assets table in PostgreSQL, or update last_seen timestamp if asset already exists

Table 2.1 continued from previous page

#	Event	Source	Input Data	System Response
2	Asset cannot be classified as IT or OT during packet analysis	PassiveMonitor	Unrecognised packet or unknown protocol	Flag asset as <i>Unknown</i> ; persist to assets table with asset_type = unknown; raise alert on Dashboard for manual SOC Analyst classification
3	New CVE entry published in NVD feed	Vulnerability Feed Collector	CVE record (CVE ID, CVSS score, description, affected products)	Correlate CVE with affected assets in inventory; store new record in vulnerabilities table; trigger contextual risk re-scoring for all affected asset-vulnerability pairs
4	EPSS score updated for an existing CVE	Vulnerability Feed Collector	Updated EPSS probability score for a CVE ID	Update epss_score field in vulnerabilities table; trigger risk re-scoring for all asset-vulnerability pairs associated with that CVE
5	CISA KEV dataset updated with a newly confirmed exploited CVE	Vulnerability Feed Collector	Updated CISA KEV list	Set cisa_kev_flag = true for all matching vulnerability records; immediately re-prioritise affected risk scores as these represent actively exploited vulnerabilities

Table 2.1 continued from previous page

#	Event	Source	Input Data	System Response
6	Contextual risk scoring triggered for an asset-vulnerability pair	RiskScoring Engine	Vulnerability record + Asset record from PostgreSQL	Build feature vector (CVSS, EPSS, CISA KEV flag, asset criticality, OT CIA weights); run trained Random Forest / XGBoost model; classify priority (Critical / High / Medium / Low); persist RiskScore to risk_scores table; forward to PolicyGatekeeper for validation
7	Policy validation requested for a computed risk score or discovered attack path	PolicyGatekeeper	RiskScore object or AttackPath object	Evaluate item against all active OT safety constraints, regulatory compliance rules, and IT/OT boundary rules; mark item as <i>Compliant</i> or <i>Flagged</i> ; if flagged, generate a PolicyViolation record and hold the item from the Dashboard pending review
8	A flagged item is resubmitted to PolicyGatekeeper after priority adjustment	PolicyGatekeeper	Adjusted RiskScore or AttackPath	Re-evaluate all active policy rules against the adjusted item; if all checks pass, mark as <i>Compliant</i> and allow forwarding to the next pipeline stage

Table 2.1 continued from previous page

#	Event	Source	Input Data	System Response
9	Attack path analysis triggered after risk scoring cycle completes	AttackPathEngine	Full asset inventory and scored vulnerability list	Construct directed NetworkX graph with assets as nodes and exploitable CVEs as edges; compute shortest-path and maximum-risk-weight paths from IT entry points to critical OT targets (PLCs, SCADA, HMIs); persist discovered paths to attack_paths table
10	SOC Analyst requests a remediation recommendation for a prioritised vulnerability	Dashboard (SOC Analyst)	Selected vulnerability ID and associated asset ID	Query RAG Vector Index for relevant policy documents, CVE advisories, and remediation templates; construct LLM prompt with retrieved context; generate OT-safe recommendation; validate output with PolicyGatekeeper; present recommendation with plain-language explanation on Dashboard
11	LLM-generated recommendation found to violate an active policy	PolicyGatekeeper	Remediation Record flagged as non-compliant	Reject the recommendation; generate a PolicyViolation record; notify Dashboard of the failure reason; prompt RemediationAdvisor to regenerate a policy-compliant alternative recommendation

Table 2.1 continued from previous page

#	Event	Source	Input Data	System Response
12	Operator approves a remediation recommendation	Dashboard (SOC Analyst / OT Engineer)	Remediation Record + APPROVED operator decision	Set operator_decision = approved in remediation_records; update vulnerability life-cycle status to <i>Awaiting-Mitigation</i> ; log decision with timestamp
13	Operator rejects a remediation recommendation	Dashboard (SOC Analyst / OT Engineer)	Remediation Record + REJECTED operator decision	Set operator_decision = rejected; log rejection reason; prompt RemediationAdvisor to generate an alternative recommendation for analyst review
14	Security Administrator adds or updates an organisational policy rule	Security Administrator	New or updated Policy object with JSON rule definition	Validate JSON rule definition for correctness; persist updated policy to policies table; reload all active policies in PolicyGatekeeper so new rules are enforced immediately in the next validation cycle
15	LLM inference service becomes unavailable during recommendation generation	Remediation Advisor	Connection timeout or service error	Return a graceful error message to Dashboard within 5 seconds; log the failure with full error trace; allow SOC Analyst to retry the request; do not block the Dashboard interface or any other pipeline processing

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. The requirements are organized according to the system modules. Each module represents a specific feature of the proposed system and includes the functional capabilities that the system must provide.

2.2.1 Module 1: Asset Discovery and Inventory Management

Following are the requirements for module 1:

1. FR1: The system shall passively monitor the industrial IT/OT network to detect connected assets without interrupting operational processes.
2. FR2: The system shall identify network devices and record asset details including device type, IP address, and active services.
3. FR3: The system shall maintain an inventory database containing approved assets within the industrial environment.
4. FR4: The system shall compare detected assets with the approved inventory to identify unauthorized or unknown devices.
5. FR5: The system shall detect running services on assets and identify services that violate predefined security policies.
6. FR6: The system shall generate alerts when unauthorized assets or services are discovered.

2.2.2 Module 2: Context-Aware Risk Scoring Engine

Following are the requirements for module 2:

1. FR1: The system shall collect vulnerability information associated with detected assets.
2. FR2: The system shall compute contextual risk scores using CVSS severity scores, EPSS exploit probability, and asset criticality values.
3. FR3: The system shall apply a machine learning model to classify vulnerabilities into High, Medium, or Low risk categories.

4. FR4: The system shall prioritize vulnerabilities based on computed contextual risk scores.
5. FR5: The system shall update vulnerability risk scores whenever new vulnerability or asset information becomes available.

2.2.3 Module 3: AI Policy and Compliance Gatekeeper

Following are the requirements for module 3:

1. FR1: The system shall evaluate detected services and vulnerabilities against predefined organizational security policies.
2. FR2: The system shall determine whether detected services comply with industrial network security rules.
3. FR3: The system shall generate alerts when policy violations are identified.
4. FR4: The system shall provide recommendations to approve, restrict, or remediate services based on contextual risk evaluation.
5. FR5: The system shall maintain records of policy compliance evaluations for auditing purposes.

2.2.4 Module 4: Attack Path Analysis and Correlation

Following are the requirements for module 4:

1. FR1: The system shall construct graph-based models representing relationships between network assets and vulnerabilities.
2. FR2: The system shall identify potential multi-step attack paths linking vulnerabilities across IT and OT systems.
3. FR3: The system shall correlate vulnerabilities to determine possible chained exploitation scenarios.
4. FR4: The system shall limit attack path depth to maintain clarity and interpretability of attack graphs.
5. FR5: The system shall provide attack path information for further analysis and visualization.

2.2.5 Module 5: Web Dashboard and Reporting Interface

Following are the requirements for module 5:

1. FR1: The system shall provide a web-based dashboard for visualizing vulnerability risks and system status.
2. FR2: The system shall display vulnerabilities ranked according to calculated contextual risk scores.
3. FR3: The system shall present visual representations of detected attack paths and vulnerability correlations.
4. FR4: The system shall display policy violations and recommended remediation actions.
5. FR5: The system shall allow users to review summarized reports of vulnerabilities, risks, and compliance status.

2.3 Non-Functional Requirements

2.3.1 Reliability

REL-1: The system shall achieve a system uptime of at least 99.5 percent per month, excluding scheduled maintenance.

REL-2: The system shall detect, log, and generate alerts for 100 percent of critical system failures within 5 seconds of occurrence.

REL-3: The system shall ensure zero data loss for asset inventory and vulnerability records by performing automatic backups every 10 minutes.

REL-4: The system shall automatically recover from at least 90 percent of non-critical failures within 30 seconds without manual intervention.

REL-5: In case of system crash, the system shall restore full operational state (including asset inventory and risk scores) within 60 seconds.

2.3.2 Usability

USE-1: The system shall allow users to locate and view risk-ranked vulnerabilities within 3 clicks or less from the dashboard homepage.

USE-2: The system shall ensure that at least 90 percent of first-time users can successfully interpret vulnerability risk levels and compliance status within 5 minutes of use (validated via usability testing).

USE-3: The system shall display attack path visualizations with clear node-link diagrams, where each path contains no more than 7 steps for readability.

USE-4: The system shall present remediation recommendations in plain language summaries of no more than 150 words, along with technical details.

USE-5: The dashboard shall maintain a consistent UI response time of less than 1 second for user interactions (e.g., clicks, filters).

2.3.3 Performance

PER-1: The system shall process incoming vulnerability data and compute contextual risk scores for at least 50 assets per second.

PER-2: The system shall update the dashboard with new risk data within 2 seconds after completion of the risk scoring process.

PER-3: The system shall generate attack path analysis results within:

- ≤ 8 seconds for 100 assets
- ≤ 15 seconds for 500 assets

PER-4: The web dashboard shall fully load within 3 seconds on a 10 Mbps connection.

PER-5: The system shall support at least 100 concurrent users without degradation in response time beyond 10%.

2.3.4 Security

SEC-1: The system shall enforce role-based access control (RBAC), ensuring that 100 percent of unauthorized access attempts are blocked.

SEC-2: All sensitive data (user credentials, vulnerability data) shall be encrypted using:

- AES-256 (at rest)
- TLS 1.2 or higher (in transit)

SEC-3: The system shall log 100 percent of user actions and system events in an immutable audit log within 1 second of occurrence.

SEC-4: The system shall enforce multi-factor authentication (MFA) for all administrative and high-privilege accounts.

SEC-5: The system shall resist brute-force login attacks by:

- Locking accounts after 5 failed attempts
- Enforcing a minimum delay of 30 seconds before retry

SEC-6: The system shall ensure that no vulnerability or risk data can be modified without proper authorization, verified through access control checks on every write operation.

Chapter 3

System Overview

CAVE-OT (Context-Aware Vulnerability Engine for OT) is an AI-driven decision-support platform designed specifically for industrial IT/OT environments. The system assists security professionals in prioritizing vulnerabilities, visualizing multi-step attack paths across IT and OT network boundaries, and receiving explainable, OT-safe remediation recommendations — all without performing active scanning or disrupting critical operational systems.

Unlike conventional vulnerability management tools that rely on static CVSS scores and active probing, CAVE-OT employs passive asset discovery, contextual risk scoring using machine learning, graph-based attack path modeling via NetworkX, and a Large Language Model (LLM) enhanced with Retrieval-Augmented Generation (RAG) for intelligent remediation guidance. The system targets hybrid IT/OT manufacturing environments and is intended for use by Security Operations Center (SOC) analysts, OT engineers, system administrators, and industrial organizations whose operations are availability-critical.

A key design principle of CAVE-OT is the **human-in-the-loop** model: all remediation decisions remain under operator control, and the system never auto-patches or performs any action that could disrupt production. The platform transforms raw vulnerability data into actionable, context-aware security decisions while maintaining operational stability.

3.1 Architectural Design

The architectural design of CAVE-OT follows a modular approach to ensure scalability, maintainability, and clear separation of concerns. The system is decomposed into multiple functional modules, each responsible for a specific task such as data collection, processing, analysis, and visualization. This modular design allows independent development and testing of components while ensuring smooth interaction between them.

To better understand the system structure, an initial Box and Line Diagram is presented, followed by the finalized layered architecture.

3.1.1 Box and Line Diagram

As a first step in the design process, Figure 3.1 presents a Box and Line Diagram providing a simplified, initial representation of CAVE-OT's major subsystems and the directional data flow connecting them. This diagram captures the five main processing stages of the system at a high level: data ingestion, enrichment, risk and analysis, policy enforcement, and presentation. It is intended to give a general understanding of how and why the system was decomposed before the final architecture pattern is applied.

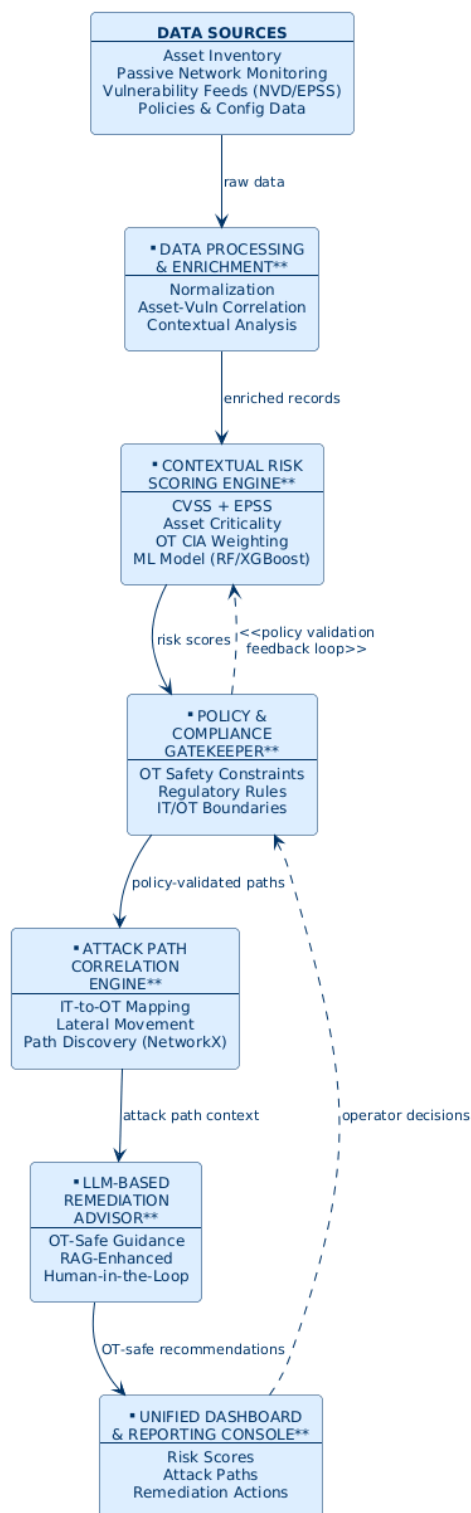


Figure 3.1: High-Level Box and Line Diagram of CAVE-OT

3.1.2 Finalized Architecture: Layered (N-Tier) Pattern

After evaluating multiple architectural patterns, the **Layered (N-Tier)** pattern was selected and finalized for CAVE-OT. As shown in Figure 3.2, the system is organized into four distinct layers — **Service Layer**, **Application Layer**, **Data Layer**, and **Presentation Layer** — each with a clearly defined responsibility. External inputs enter the system from two sources: raw packets from the IT/OT Network (containing SCADA, PLC, HMI, and Enterprise IT nodes) and CVE records, EPSS scores, and KEV flags from the Vulnerability Intelligence Feeds (NVD, EPSS, CISA KEV). Two user roles interact with the system — SOC Analyst and OT Engineer — both operating exclusively through the Presentation Layer. A controlled *Policy Violation Feedback Loop* operates between the Policy and Compliance Gatekeeper in the Application Layer and the Risk Scoring Engine, ensuring all outputs are re-evaluated against OT safety, regulatory, and boundary constraints before reaching the Presentation Layer. This pattern was chosen for the following reasons:

- **Separation of concerns:** Each layer can be developed, tested, and maintained independently. The Risk Scoring Engine in the Application Layer can be retrained without modifying the Dashboard in the Presentation Layer or the policy rules in the Gatekeeper.
- **Modularity:** New data sources, scoring algorithms, or policy rules can be introduced at the relevant layer without redesigning the entire system. The Vulnerability Feed Collector can be extended to new feeds beyond NVD, EPSS, and CISA KEV without touching any other layer.
- **OT safety:** The strictly controlled one-directional data flow prevents any automated action from propagating back to the OT network unintentionally. The Policy and Compliance Gatekeeper acts as the key control point, flagging violations and blocking non-compliant outputs before they reach the operator.
- **Testability:** Each layer exposes a clean interface to the layer above it, making unit testing and integration testing straightforward. The Data Layer can be tested in isolation using mock repositories without requiring the Application Layer to be running.

Table 3.1 maps every module and component to its corresponding software layer, and Figure 3.2 shows the finalized architecture diagram with all components placed visually within their respective layers.

Table 3.1: Module-to-Layer Mapping (4-Layer Software Architecture)

Software Layer	Responsibility	Modules / Components
Service Layer — Data Ingestion & Enrichment	Passive network sensing, asset classification, vulnerability feed collection, and asset-vulnerability correlation	Passive Asset Discovery (Scapy / PCAP); Audit-Based Inventory (Asset classification); Vulnerability Feed Collector (NVD, EPSS, CISA KEV); Asset-Vulnerability Correlator (Maps CVEs to assets)
Application Layer — Core AI Intelligence	Core business logic, risk scoring, attack path analysis, policy enforcement, and remediation advisory	Risk Scoring Engine (Random Forest / XGBoost, CVSS + EPSS + OT CIA weighting); Attack Path Engine (NetworkX directed graph, IT-to-OT lateral movement); Policy & Compliance Gatekeeper (OT safety, regulatory, boundary checks, flags violations); LLM Remediation Advisor (RAG-enhanced, OT-safe recommendations, Human-in-the-Loop)
Data Layer — Persistent Storage	Persistent storage of all system entities including assets, vulnerabilities, risk scores, policies, graph topology, and remediation templates	PostgreSQL (Assets, Vulnerabilities, Risk scores, Policies, Audit logs, Users); Neo4j Graph DB (Network topology nodes, Attack path edges); RAG Vector Index (Policy docs, CVE advisories, Remediation templates)
Presentation Layer — Web Dashboard	User interface, operator decision support, result display, and compliance reporting	Unified Dashboard (Risks, Attack paths, Alerts); Human-in-the-Loop Interface (Operator approve / reject decisions); Reporting Console (Compliance and executive reports)

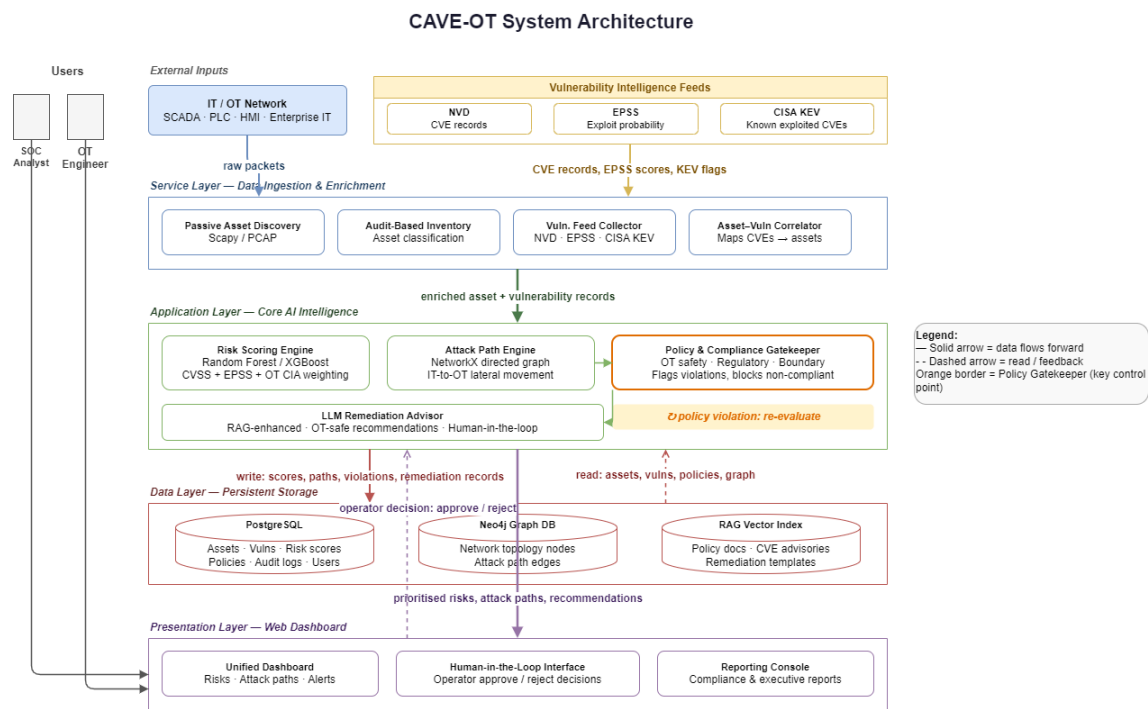


Figure 3.2: Finalized 4-Layer Software Architecture Diagram of CAVE-OT

3.2 Data Design

3.2.1 Overview

The information domain of CAVE-OT encompasses three core entities: discovered network assets, mapped vulnerabilities, and security alerts generated by the Suricata intrusion detection system. These entities are transformed from raw network observations and external threat intelligence sources into structured relational data stored in a PostgreSQL database named `cave_ot`.

The data layer is managed through a dedicated Python module, `db_manager.py`, which abstracts all database operations and ensures that no other component in the system writes SQL directly.

3.2.2 Data Entities and Storage

3.2.2.1 Assets

The `assets` table represents every OT or IT device discovered on the monitored network. Each asset is uniquely identified by the combination of its IP address and port number, enforced through a `UNIQUE` constraint on `(ip, port)`.

This design reflects industrial environments where multiple services may exist on different ports of the same device.

Table 3.2: Assets Table Schema

Column	Type	Description
id	SERIAL	Auto-incremented primary key
ip	VARCHAR(45)	Device IP address supporting IPv4 and IPv6
port	INTEGER	Service port number
service	VARCHAR(50)	Industrial protocol such as Modbus, S7comm, DNP3, or BACnet
device_type	VARCHAR(50)	Functional device type such as PLC, RTU, HMI, or Sensor
zone	VARCHAR(10)	Network zone classification: OT or IT
vendor	VARCHAR(100)	Device manufacturer
product	VARCHAR(100)	Product or model name
firmware	VARCHAR(50)	Firmware version string
criticality	FLOAT	Operational criticality score ranging from 0.0 to 1.0
packet_count	INTEGER	Number of packets observed during passive discovery
first_seen	TIMESTAMP	Timestamp of initial discovery
last_seen	TIMESTAMP	Timestamp of latest observation
status	VARCHAR(10)	Asset status classified as ACTIVE or INACTIVE

The criticality field is a domain-specific attribute that quantifies the operational importance of a device within the industrial process.

For example, a chemical dosing pump PLC may be assigned a criticality value of 0.97, whereas a ventilation controller may receive a value of 0.55. This value directly influences the contextual risk score associated with the device vulnerabilities.

The status field is updated automatically after every pipeline execution. An asset is marked as ACTIVE if its last_seen timestamp falls within the previous ten minutes; otherwise, it is marked as INACTIVE.

This mechanism enables passive real-time network presence tracking without performing active probing.

3.2.2.2 Vulnerabilities

The vulnerabilities table stores CVE records mapped to individual assets.

A many-to-one relationship exists between vulnerabilities and assets through the `asset_id` foreign key. Cascading deletion ensures referential integrity.

A composite UNIQUE constraint on (`asset_id`, `cve_id`) prevents duplicate CVE mappings.

Table 3.3: Vulnerabilities Table Schema

Column	Type	Description
<code>id</code>	SERIAL	Auto-incremented primary key
<code>asset_id</code>	INTEGER	Foreign key referencing <code>assets(id)</code>
<code>cve_id</code>	VARCHAR(20)	CVE identifier such as CVE-2021-44228
<code>cvss</code>	FLOAT	CVSS base score from NVD ranging from 0.0 to 10.0
<code>epss</code>	FLOAT	EPSS exploitation probability score
<code>kev</code>	BOOLEAN	Indicates whether the CVE exists in the CISA KEV catalog
<code>c_impact</code>	FLOAT	Confidentiality impact score
<code>i_impact</code>	FLOAT	Integrity impact score
<code>a_impact</code>	FLOAT	Availability impact score
<code>similarity</code>	FLOAT	TF-IDF cosine similarity score generated during CVE mapping
<code>risk_score</code>	FLOAT	Context-aware computed risk score ranging from 0.0 to 10.0
<code>risk_tier</code>	VARCHAR(10)	Severity classification: CRITICAL, HIGH, MEDIUM, or LOW
<code>discovered_at</code>	TIMESTAMP	Timestamp when the CVE was first mapped to the asset
<code>updated_at</code>	TIMESTAMP	Timestamp of the most recent risk score update

The `risk_score` field does not store the raw CVSS value. Instead, it represents a con-

textual risk score calculated using multiple factors including:

- CVSS base score
- EPSS exploitation probability
- CISA KEV exploitation status
- OT-specific CIA weighting
- Asset operational criticality

The OT CIA weighting is defined as follows:

- Availability = 0.50
- Integrity = 0.30
- Confidentiality = 0.20

This produces a contextual risk score that more accurately reflects real-world industrial exploitability.

3.2.2.3 Alerts

The `alerts` table stores security events generated by the Suricata intrusion detection system.

Each alert is linked to an asset through `asset_id` and may optionally reference a vulnerability using `vulnerability_id`. This relationship enables correlation between detected attack traffic and known vulnerabilities.

Table 3.4: Alerts Table Schema

Column	Type	Description
id	SERIAL	Auto-incremented primary key
asset_id	INTEGER	Foreign key referencing assets(id)
vulnerability_id	INTEGER	Optional foreign key referencing vulnerabilities(id)
alert_signature	VARCHAR(255)	Suricata rule signature name
alert_category	VARCHAR(100)	Alert classification category
severity	INTEGER	Severity level where 1=Critical, 2=High, and 3=Medium
protocol	VARCHAR(20)	Associated network protocol such as Modbus or HTTP
src_ip	VARCHAR(45)	Source IP address
dst_ip	VARCHAR(45)	Destination IP address
src_port	INTEGER	Source port number
dst_port	INTEGER	Destination port number
is_active_attack	BOOLEAN	Indicates confirmed active exploitation
alert_count	INTEGER	Number of repeated alert occurrences
raw_event	TEXT	Full JSON payload extracted from Suricata eve.json
detected_at	TIMESTAMP	Timestamp of alert detection

3.2.3 Entity Relationship

The three tables form a hierarchical relational structure.

An asset may contain zero or more vulnerabilities, and each vulnerability may contain zero or more associated alerts.

Cascading deletion is applied on the foreign key relationships to maintain database consistency.

```
assets (1)
|
|-- vulnerabilities (many)
|       |
|       |-- alerts (many)
|
|-- alerts (many)
```

Figure 3.3: Entity Relationship Structure of the CAVE-OT Database

3.2.4 Data Processing and Transformation

Raw network observations are captured as JSON by the passive discovery pipeline implemented in `discover.py` and stored in `vulnerability_scan_results.json`.

The database ingestion module, `db_ingestor.py`, processes this intermediate file before persistence into PostgreSQL.

The transformation process performs the following operations:

1. Format Normalisation

The ingestor processes multiple JSON input formats including `assets.json`, `model_input.json`, and `vulnerability_scan_results.json`, converting them into a uniform device structure.

2. Risk Score Enrichment

Any CVE record lacking a `risk_score` field is enriched using the contextual risk scoring formula before insertion.

3. UPSERT Logic

Assets and vulnerabilities are inserted using PostgreSQL `INSERT ... ON CONFLICT DO UPDATE` statements to avoid duplicate records.

4. Status Classification

Asset statuses are recalculated after each execution using the `last_seen` timestamp.

3.2.5 Indexing Strategy

Seven database indexes are implemented to optimise query performance for the dashboard and REST API layer.

- `idx_assets_status` — filters ACTIVE and INACTIVE assets
- `idx_assets_ip` — supports IP-based asset lookup
- `idx_vuln_asset_id` — supports vulnerability joins
- `idx_vuln_risk_tier` — filters vulnerabilities by tier
- `idx_vuln_risk_score` — sorts vulnerabilities by risk score
- `idx_alerts_asset_id` — supports alert joins
- `idx_alerts_severity` — filters alerts by severity

3.2.6 Intermediate Data Structures

In addition to the relational database, the system uses intermediate file-based structures during pipeline execution.

- **TF-IDF Model Files (.pkl)**

Six serialized Python objects stored in the `model/` directory contain trained TF-IDF vectorizers, sparse matrices, and CVE dataframes for both the general and OT-specific datasets.

- **Scan Result JSON** (`vulnerability_scan_results.json`)

This intermediate file stores the complete discovered device inventory and mapped CVEs before database ingestion.

3.3 Domain Model

The domain model for CAVE-OT captures the key real-world concepts of the industrial IT/OT vulnerability management domain and the relationships between them, independent of any implementation or technology detail. The central concept is the Asset — any IT or OT device present in the industrial network. Assets are the subjects of Vulnerabilities (CVE records) that are continuously supplied by external Vulnerability Intelligence Feeds (NVD, EPSS, CISA KEV) and correlated with the asset inventory. Each vulnerability is evaluated to produce a Risk Score that combines exploit severity, likelihood, and the operational context of the affected asset. Vulnerabilities can be chained across multiple assets to form Attack Paths that cross IT and OT network boundaries, representing real multi-step lateral movement threats. All risk scores and attack paths are evaluated against Policies defined by the organisation to enforce OT safety, regulatory

compliance, and boundary constraints. When a vulnerability warrants action, the system produces a Remediation Record containing an OT-safe recommendation that must be explicitly approved by a human Operator before any remediation step is taken, embodying the human-in-the-loop principle at the heart of CAVE-OT.

Figure 3.4 presents the domain model for CAVE-OT.

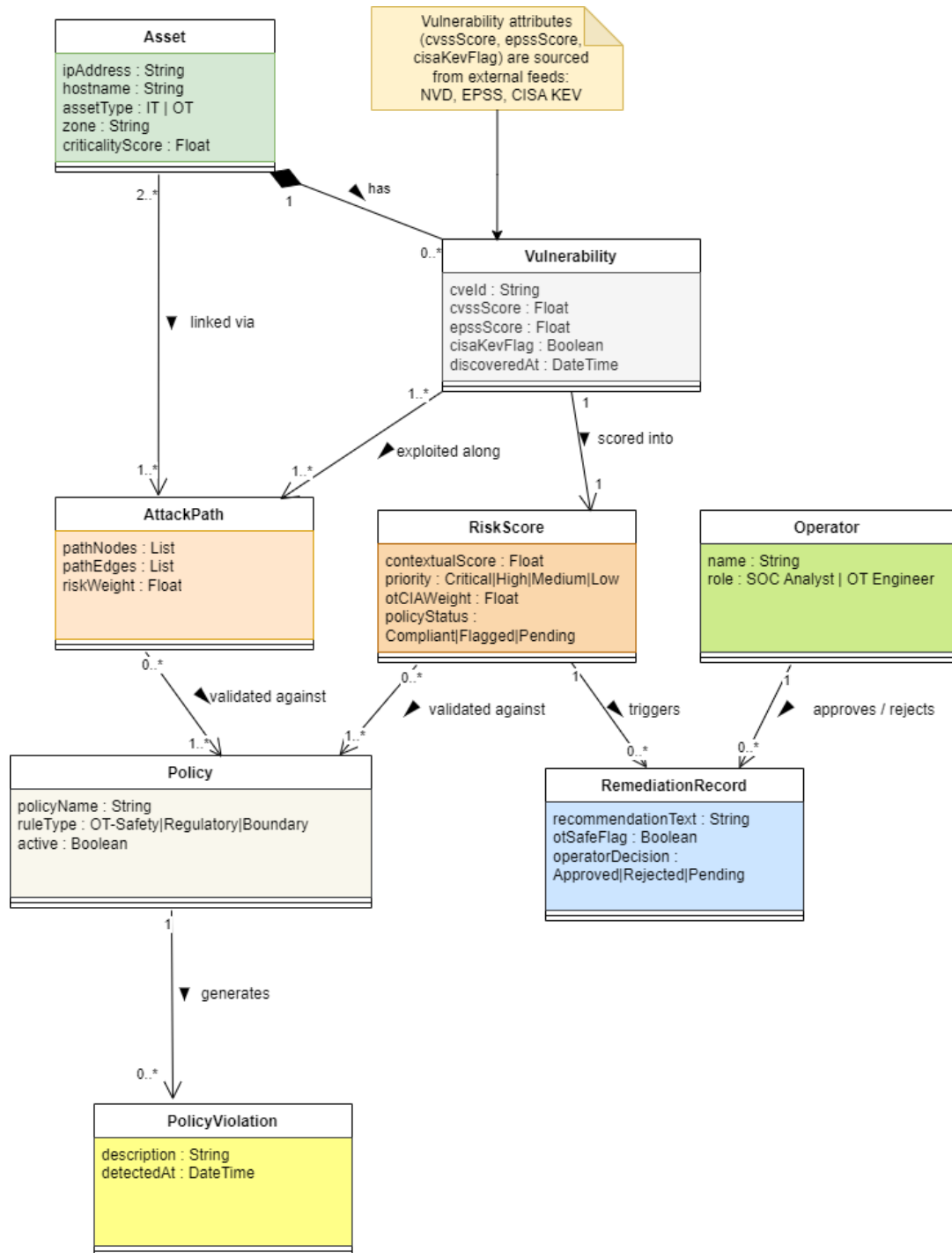


Figure 3.4: Domain Model for CAVE-OT

3.4 Design Models

The following design models describe the structure and behaviour of the CAVE-OT system as implemented in the current iteration. The system follows a pipeline-oriented architecture comprising passive network asset discovery, CVE mapping using TF-IDF cosine similarity, contextual risk scoring with policy compliance gating, PostgreSQL database persistence, and a Flask-based monitoring dashboard.

3.4.1 Activity Diagram

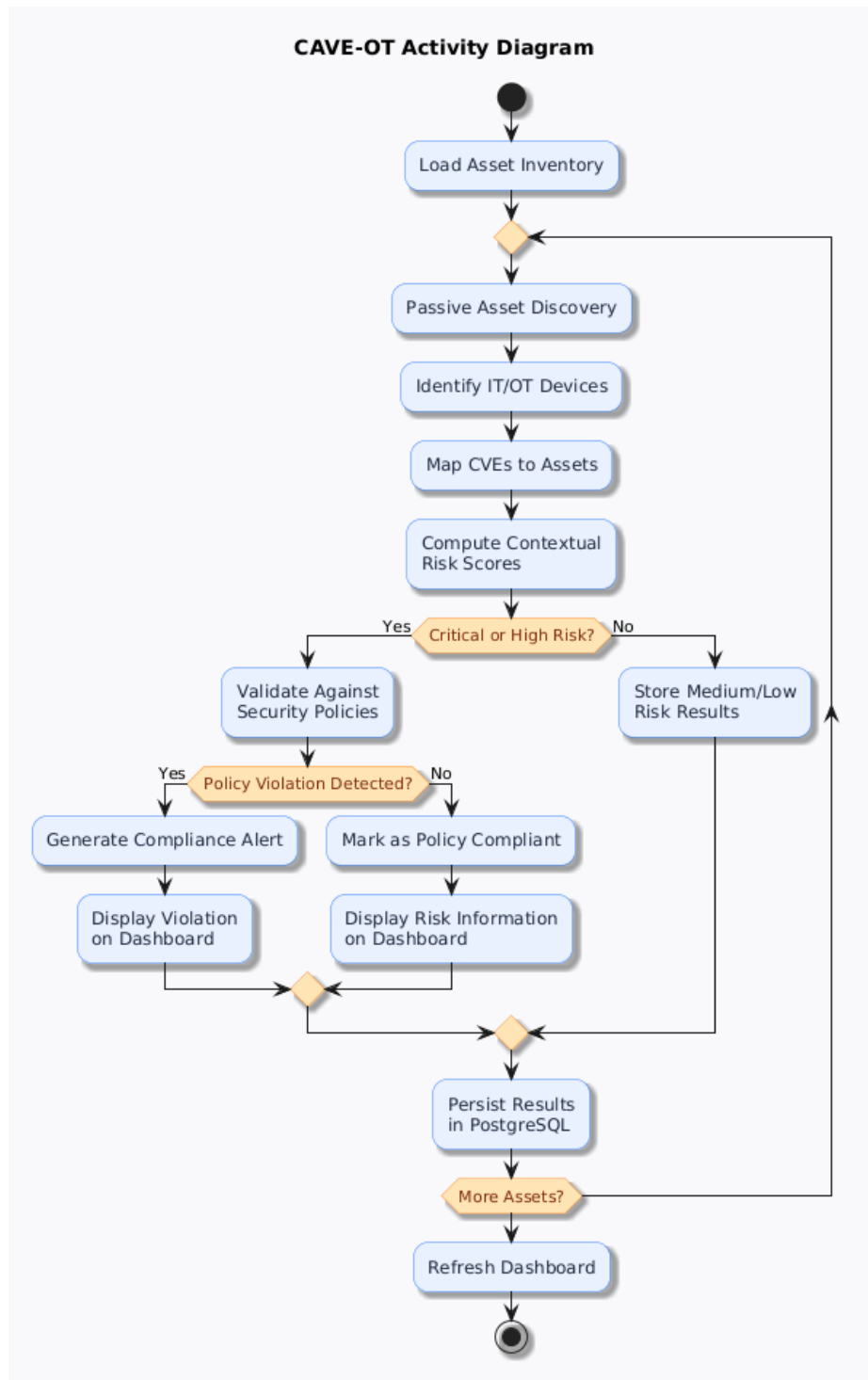


Figure 3.5: Activity Diagram — CAVE-OT Pipeline End-to-End Flow

Figure 3.5 illustrates the complete operational flow of the CAVE-OT system from initial asset loading through to dashboard rendering. The diagram uses rounded rectangle nodes for actions, orange diamond nodes for decision points, and guard conditions labelled *Yes* and *No* on transition arrows.

The pipeline begins when the system loads the asset inventory, reading all OT and IT device records. The flow then enters the main processing loop. For each asset, the system performs **Passive Asset Discovery** to detect the device on the network, followed by **Identify IT/OT Devices** to classify the device type and zone. The **Map CVEs to Assets** step uses the trained TF-IDF cosine similarity model to retrieve the top-25 most relevant CVEs for each device. The **Compute Contextual Risk Scores** step then applies the scoring formula combining CVSS, EPSS, KEV status, OT-specific CIA reweighting, and asset criticality to produce a final risk score between 0.0 and 10.0.

A decision node labelled **Critical or High Risk?** branches the flow. If the CVE is classified as Critical or High (score ≥ 6.0), the flow proceeds to **Validate Against Security Policies**, where the Policy Compliance Gatekeeper evaluates the CVE against defined thresholds. A second decision node **Policy Violation Detected?** further branches the flow: if a violation is detected, the system **Generates a Compliance Alert** and **Displays the Violation on the Dashboard**; if no violation is detected, the CVE is **Marked as Policy Compliant** and **Risk Information is Displayed on the Dashboard**. Both paths converge at a merge node. If the CVE is Medium or Low risk (score < 6.0), the flow takes the *No* branch directly to **Store Medium/Low Risk Results**, which also converges at the same merge node.

After convergence, the system **Persists Results in PostgreSQL** using UPSERT operations. A final decision node **More Assets?** checks whether additional devices remain in the inventory. If yes, control returns to the top of the loop. If all assets have been processed, the flow proceeds to **Refresh Dashboard**, which queries PostgreSQL through the Flask REST API and renders the updated monitoring interface, ending the flow at the terminal state.

3.4.2 Class Diagram

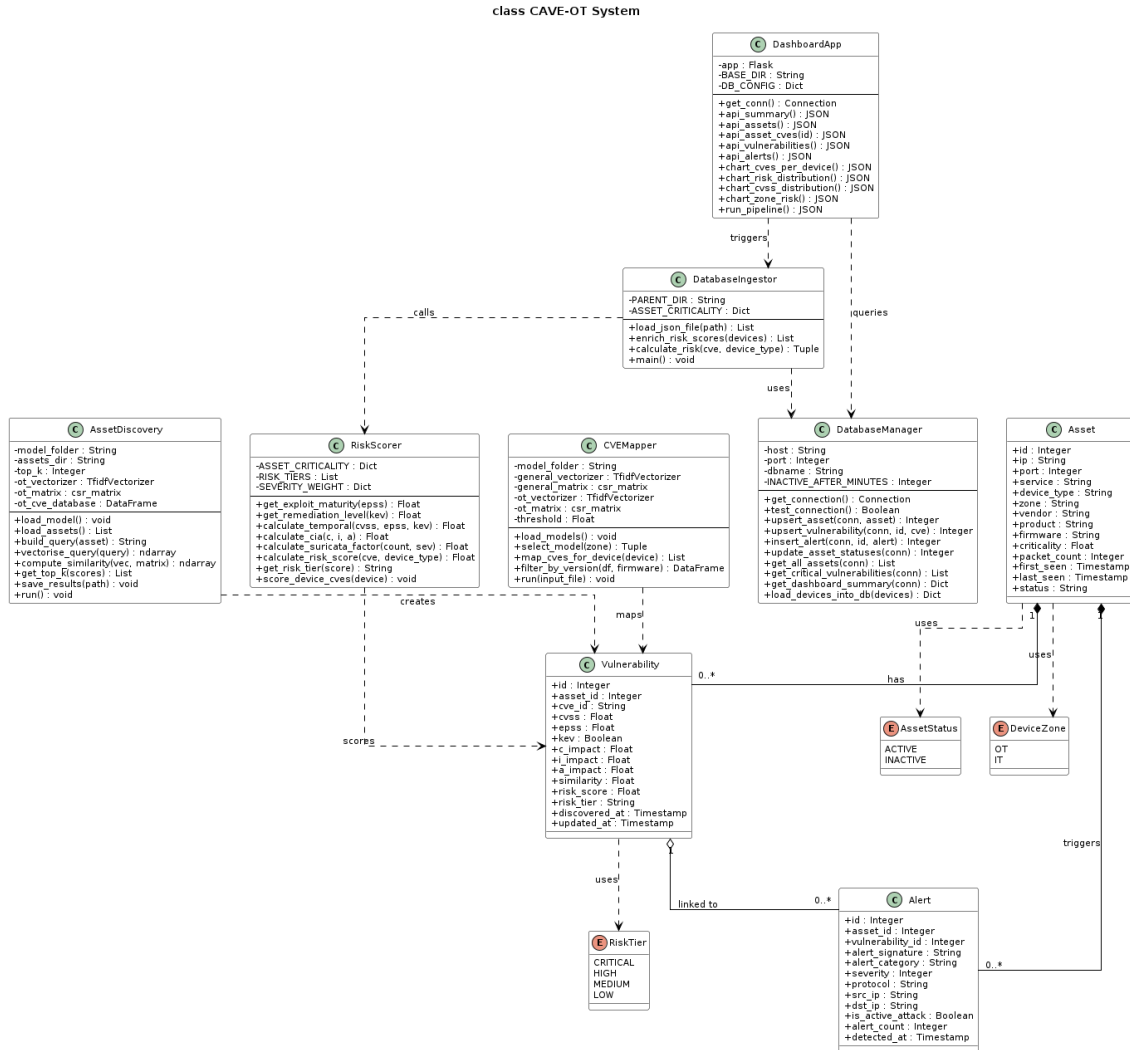


Figure 3.6: Class Diagram — CAVE-OT System Components and Relationships

Figure 3.6 presents the class diagram for the CAVE-OT system, modelling the six functional component classes, three data entity classes, and three enumeration classes, along with their attributes, methods, and inter-class relationships.

DashboardApp sits at the top of the hierarchy as the entry point of the system. It is the Flask web application that serves five HTML pages and exposes ten REST API endpoints including `api_summary()`, `api_assets()`, `api_vulnerabilities()`, `api_alerts()`, and `run_pipeline()`. It holds private attributes for the Flask application instance, the base directory path, and the database configuration dictionary. It *triggers* `DatabaseIngestor` and *queries* `DatabaseManager` directly for all data retrieval.

DatabaseIngestor acts as the bridge between the pipeline output and the database. Its private attributes include the parent directory path and the asset criticality dictionary. Its public methods normalise three distinct JSON input formats (`assets.json`, `model_input.json`, and `vulnerability_scan_results.json`) into a uniform device list, enrich any CVE records that lack risk scores by calling the scoring formula inline, and delegate all SQL operations to `DatabaseManager`. It *calls* `AssetDiscovery` and *uses* `DatabaseManager`.

DatabaseManager is the sole class that interacts directly with PostgreSQL. Its private attributes hold the host, port, database name, user credentials, and the inactivity threshold of ten minutes. Its public methods include `upsert_asset()`, `upsert_vulnerability()`, `insert_alert()`, `update_asset_statuses()`, `get_all_assets()`, `get_critical_vulnerabilities()`, `get_dashboard_summary()`, and `load_devices_into_db()`. All other components that require database access delegate to this class.

AssetDiscovery encapsulates the passive scanning logic of `discover.py`. Its private attributes include the model folder path, the assets directory path, the top-K value set to 25, and the three loaded TF-IDF model objects: the vectoriser, the sparse matrix, and the CVE dataframe. Its public methods cover the full pipeline from loading the model and asset inventory, building the query string, vectorising the query, computing cosine similarity, retrieving the top-K matches, saving results to JSON, and running the complete scan. It *creates* `Vulnerability` records.

RiskScorer is a stateless computation class whose methods implement the CVSS v3.1 temporal and environmental scoring formula with OT-specific CIA reweighting. Its private attributes are the asset criticality dictionary, the risk tier threshold list, and the Suricata severity weight dictionary. Its public methods include `get_exploit_maturity()`, `get_remediation_level()`, `calculate_temporal()`, `calculate_cia()`, `calculate_suricata_factor()`, `calculate_risk_score()`, `get_risk_tier()`, and `score_device_cves()`. It *scores* `Vulnerability` records.

CVEMapper implements a more detailed mapping strategy in `cve_mapper.py`. Unlike `AssetDiscovery`, it holds both the general model (206,553 CVEs) and the OT-specific model (4,792 CVEs) and selects between them based on the device zone. It addition-

ally filters CVE candidates by firmware version range using the `version_start` and `version_end` fields from the NVD dataset, and applies a similarity threshold to remove low-confidence matches. It *maps* Vulnerability records.

The three data entity classes correspond directly to the three PostgreSQL tables. **Asset** holds all device attributes including IP address, port, service, device type, zone, vendor, product, firmware, criticality, packet count, timestamps, and status. It has a one-to-many composition relationship with **Vulnerability**: an asset owns its vulnerabilities and deleting an asset cascades to all its CVE records. **Vulnerability** holds the CVE identifier, CVSS, EPSS, KEV flag, CIA impact scores, similarity score, risk score, risk tier, and timestamps. Both **Asset** and **Vulnerability** have one-to-many relationships with **Alert**, which stores Suricata-generated security events. Three enumeration classes — **RiskTier** (CRITICAL, HIGH, MEDIUM, LOW), **AssetStatus** (ACTIVE, INACTIVE), and **DeviceZone** (OT, IT) — define the constrained value sets used across the system.

3.4.3 Class-Level Sequence Diagram

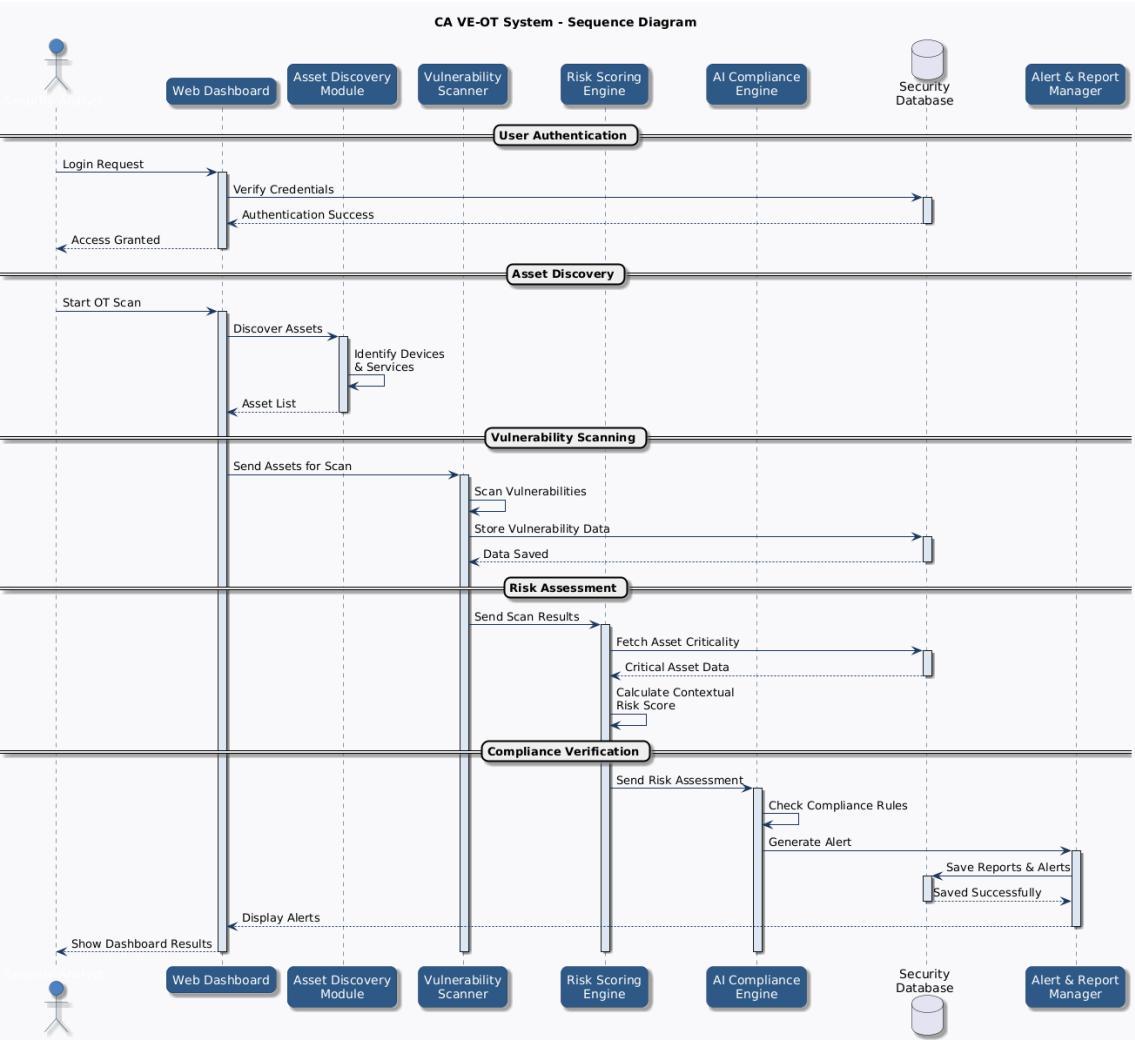


Figure 3.7: System-Level Sequence Diagram — CAVE-OT End-to-End Interaction Flow

Figure 3.7 presents the system-level sequence diagram for the CAVE-OT system, illustrating the complete message flow between the *Security Analyst*, the *Web Dashboard*, the *Asset Discovery Module*, the *Vulnerability Scanner*, the *Risk Scoring Engine*, the *AI Compliance Engine*, the *Security Database*, and the *Alert & Report Manager*. The diagram is divided into five interaction frames, each representing a distinct operational phase of the system.

Frame 1 — User Authentication. The sequence begins when the Security Analyst submits a Login Request to the *Web Dashboard*. The dashboard forwards a Verify Credentials message to the *Security Database*, which validates the analyst’s identity and returns an Authentication Success response. The dashboard then grants access to the analyst with an Access Granted message, completing the authentication phase and enabling the analyst to interact with the system.

Frame 2 — Asset Discovery. Once authenticated, the analyst initiates a scan by sending a Start OT Scan message to the *Web Dashboard*. The dashboard forwards a Discover Assets message to the *Asset Discovery Module*. The module performs passive network discovery by sending Identify Devices & Services to itself, detecting OT and IT devices on the monitored network including PLCs, RTUs, HMIs, and sensors along with their associated industrial protocols such as Modbus, S7comm, DNP3, and BACnet. The module returns an Asset List to the dashboard, which presents the discovered inventory to the analyst.

Frame 3 — Vulnerability Scanning. The dashboard forwards the discovered asset list to the *Vulnerability Scanner* via a Send Assets for Scan message. The scanner performs TF-IDF cosine similarity matching against the NVD CVE corpus, sending a Scan Vulnerabilities message to itself to retrieve the top-25 most relevant CVEs per device. The scanner then sends Store Vulnerability Data to the *Security Database* to persist the raw CVE mappings. The database confirms persistence with a Data Saved response, completing the vulnerability scanning phase.

Frame 4 — Risk Assessment. The *Vulnerability Scanner* forwards the scan results to the *Risk Scoring Engine* via a Send Scan Results message. The engine sends a Fetch Asset Criticality message to the *Security Database* to retrieve the criticality values associated with each discovered asset (PLC = 1.00, RTU = 0.95, HMI = 0.80 as defined in IEC 62443). The database returns the Critical Asset Data. The engine then computes the contextual risk score for each CVE by applying the CVSS v3.1 temporal scoring formula with OT-specific CIA reweighting (Availability: 0.50, Integrity: 0.30, Confidentiality: 0.20 per NIST SP 800-82) and the asset criticality factor, sending a Calculate Contextual Risk Score message to itself and returning the scored results.

Frame 5 — Compliance Verification. The *Risk Scoring Engine* forwards the risk assessment results to the *AI Compliance Engine* via a Send Risk Assessment message.

The compliance engine evaluates each CVE against the defined security policy rules by sending a `Check Compliance Rules` message to itself. Policy rules include a risk score threshold of 8.0 for CRITICAL classification, a KEV flag indicating confirmed exploitation in the wild, an EPSS probability exceeding 0.10 on a critical asset, and an asset criticality value of 0.90 or above. For any CVE that violates the policy, the engine sends a `Generate Alert` message, which triggers the *Alert & Report Manager* to `Save Reports & Alerts` to the *Security Database*. The database confirms with a `Saved Successfully` response. The compliance engine then sends `Display Alerts` to the *Web Dashboard*, which presents the final `Show Dashboard Results` to the Security Analyst, completing the end-to-end pipeline and rendering the monitoring interface with all asset states, vulnerability distributions, risk tiers, and compliance alerts.

3.4.4 State Transition Diagram

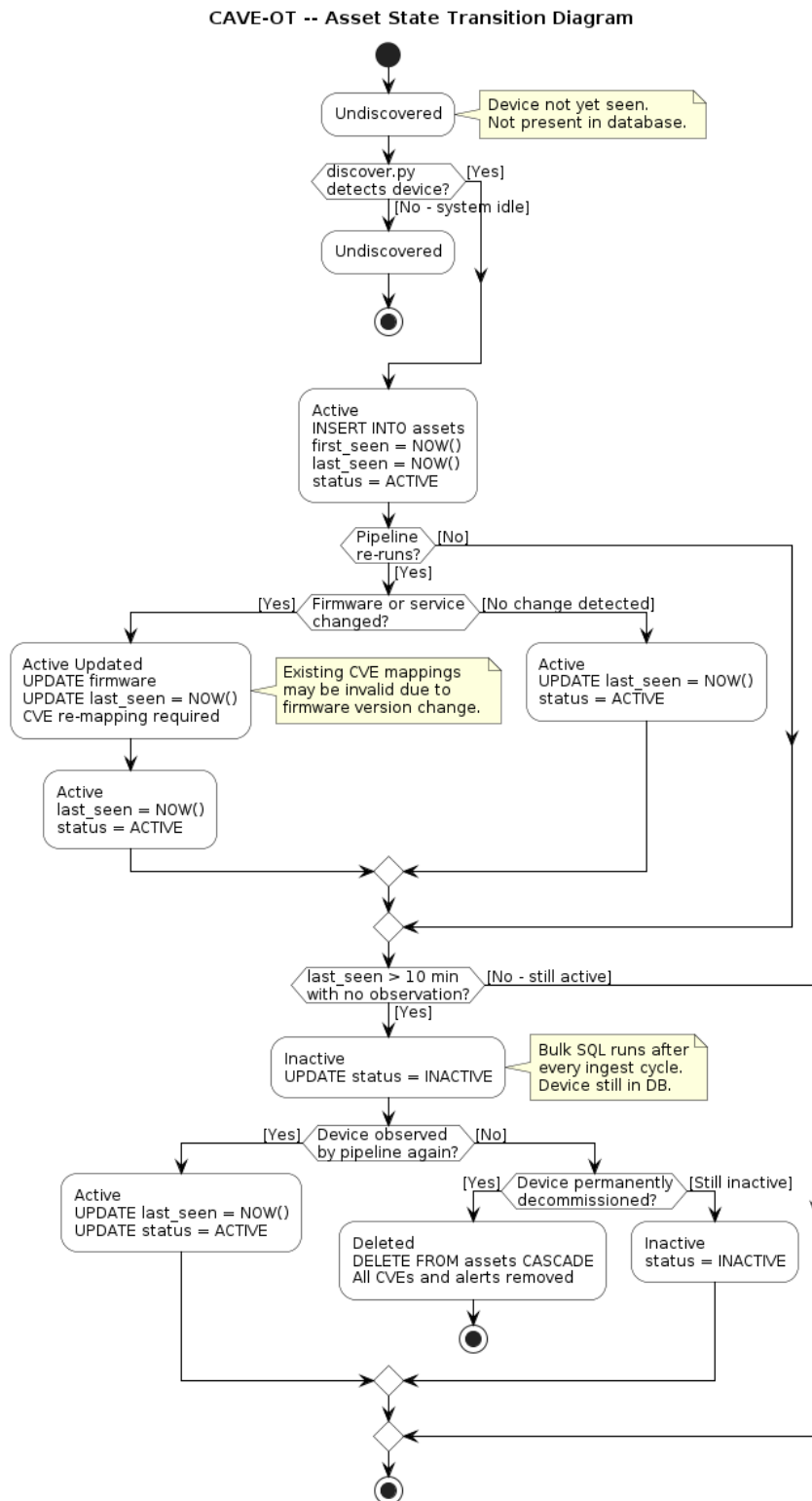


Figure 3.8: State Transition Diagram — Asset Entity Lifecycle in CAVE-OT

Figure 3.8 models the lifecycle of an Asset entity within the CAVE-OT database. The diagram uses rounded rectangle nodes for states, diamond nodes for decision points, and guard conditions in square brackets on transition arrows. The filled circle represents the initial pseudo-state and the double circle represents the terminal state. Notes attached with dashed lines provide additional context for key transitions.

The lifecycle begins at the **Undiscovered** state. A note on the diagram confirms that the device is not yet seen and not present in the database. A decision node labelled *discover.py detects device?* branches the flow. If the system is idle and no device is detected, the guard condition *[No — system idle]* keeps the asset in the Undiscovered state and the flow reaches a local terminal. If the device is detected, the guard condition *[Yes]* fires and the asset transitions to the **Active** state, executing an `INSERT INTO assets` statement with `first_seen = NOW()`, `last_seen = NOW()`, and `status = ACTIVE`.

From the **Active** state, a decision node *Pipeline re-runs?* branches the flow. If the pipeline does not re-run, the guard condition *[No]* leads directly to the inactivity check. If the pipeline re-runs, a second decision node *Firmware or service changed?* further branches the flow. If a change is detected, the guard condition *[Yes]* transitions the asset to the **Active Updated** state, executing `UPDATE firmware`, `UPDATE last_seen = NOW()`, and noting that CVE re-mapping is required. A note on the diagram states that existing CVE mappings may be invalid due to the firmware version change. The asset then transitions back to **Active** with `last_seen = NOW()` and `status = ACTIVE`. If no change is detected, the guard condition *[No change detected]* transitions the asset to a refreshed **Active** state with only `last_seen = NOW()` updated.

Both paths from the Active state converge at a merge node, which feeds into the inactivity decision node *last_seen > 10 min with no observation?*. If the guard condition *[No — still active]* is satisfied, the asset remains active and the flow loops back. If the guard condition *[Yes]* is satisfied, the asset transitions to the **Inactive** state, executing `UPDATE status = INACTIVE`. A note on the diagram confirms that this bulk SQL runs after every ingest cycle and that the device remains in the database.

From the **Inactive** state, a decision node *Device observed by pipeline again?* branches the flow. If the device is observed again, the guard condition *[Yes]* transitions the asset back to **Active**, executing `UPDATE last_seen = NOW()` and `UPDATE status = ACTIVE`. If the device is not observed, a further decision node *Device permanently decommissioned?* branches the flow. If the guard condition *[Yes]* is satisfied, the asset transitions to the **Deleted** state, executing `DELETE FROM assets CASCADE` and removing all associated CVE records and alerts, reaching the terminal state. If the guard condition *[Still inactive]* is satisfied, the asset remains in the **Inactive** state with `status = INACTIVE`. The two remaining merge nodes at the bottom of the diagram consolidate all inactive and deleted paths before reaching the final terminal state.

The SQL that implements the inactivity transition is:

```
UPDATE assets SET status = CASE
    WHEN last_seen >= NOW() - INTERVAL '10 minutes'
    THEN 'ACTIVE'
    ELSE 'INACTIVE'
END;
```


Chapter 4

Implementation and Testing

CAVE-OT (Context-Aware Vulnerability Engine for OT Systems) is a passive vulnerability management system designed for industrial OT/ICS environments. The system addresses a fundamental limitation of conventional vulnerability management tools, which rely on static CVSS scores and active network scanning — both of which are unsuitable for operational technology environments where availability is critical and active probing can disrupt physical processes.

The current iteration implements the complete data acquisition and risk prioritisation pipeline. Network assets are discovered passively from a device inventory, matched to known CVEs using a trained TF-IDF cosine similarity model, scored using a contextual risk formula that incorporates CVSS, EPSS, CISA KEV status, OT-specific CIA reweighting, and asset criticality, and persisted to a PostgreSQL database. A Flask-based monitoring dashboard provides real-time visibility into asset states, vulnerability distributions, and risk tiers.

4.1 Algorithm Design

CAVE-OT implements three core algorithms to deliver its vulnerability management pipeline. The **TF-IDF Cosine Similarity** algorithm maps each discovered OT device to its most relevant CVEs by treating the device description as a query against a trained corpus of over 165,000 general CVEs and 4,700 OT-specific CVEs. The **Contextual Risk Scoring** algorithm replaces static CVSS prioritisation with a multi-factor formula that combines CVSS v3.1 temporal scoring, EPSS exploit probability, CISA KEV confirmation, OT-specific CIA reweighting aligned with NIST SP 800-82, asset criticality coefficients from IEC 62443, and live Suricata network alert context. The **UPSERT-Based Database Persistence** algorithm ensures that repeated pipeline runs update existing records rather than creating duplicates, and automatically reclassifies asset activity status based on a ten-minute observation window. The pseudocode for each algorithm is presented below.

Algorithm 1 TF-IDF Cosine Similarity CVE Mapping

Input: Asset inventory $A = \{a_1, a_2, \dots, a_n\}$; trained TF-IDF vectoriser V , matrix M , CVE database D ; top- K value where $K = 25$

Output: CVE mapping list R for each asset in A

```

1: Load  $V, M, D$  from serialised model files
2: Load asset inventory  $A$  from assets.json
3:  $R \leftarrow []$ 
4: for each asset  $a_i \in A$  do
5:    $q \leftarrow \text{CONCAT}(a_i.\text{vendor}, a_i.\text{product}, a_i.\text{firmware})$ 
6:    $q \leftarrow \text{LOWERCASE}(q)$ 
7:    $\vec{q} \leftarrow V.\text{TRANSFORM}(q)$ 
8:    $S \leftarrow \text{COSINESIMILARITY}(\vec{q}, M)$ 
9:    $I \leftarrow \text{ARGSORT}(S, \text{descending})[0 : K]$ 
10:   $C_i \leftarrow []$ 
11:  for each index  $j \in I$  do
12:    Append  $D[j]$  to  $C_i$ 
13:  end for
14:  Append  $\{\text{device: } a_i, \text{cves: } C_i\}$  to  $R$ 
15: end for
16: SAVEJSON(R, vulnerability_scan_results.json)
17: return  $R$ 

```

Figure 4.1: Algorithm 1: TF-IDF Cosine Similarity CVE Mapping

Algorithm 2 Contextual Risk Scoring for OT Vulnerabilities

Input: CVE record *cve* (cvss, epss, kev, c_impact, i_impact, a_impact); device type *dt*; alert count *ac*; alert severity *as*

Output: Risk score *score* $\in [0.0, 10.0]$; tier $\in \{\text{CRITICAL, HIGH, MEDIUM, LOW}\}$

```

1: // Step 1: Exploit Code Maturity from EPSS
2: if epss  $\geq 0.70$  then
3:   ECM  $\leftarrow 1.00$ 
4: else if epss  $\geq 0.40$  then
5:   ECM  $\leftarrow 0.97$ 
6: else if epss  $\geq 0.10$  then
7:   ECM  $\leftarrow 0.94$ 
8: else
9:   ECM  $\leftarrow 0.91$ 
10: end if
11: // Step 2: Remediation Level from CISA KEV
12: if kev = 1 then
13:   RL  $\leftarrow 1.00$ 
14: else
15:   RL  $\leftarrow 0.95$ 
16: end if
17: // Step 3: Temporal score
18: T  $\leftarrow \text{cvss} \times \text{ECM} \times \text{RL}$ 
19: // Step 4: OT CIA reweighting (Availability prioritised)
20: CIA  $\leftarrow (\text{c\_impact} \times 0.20) + (\text{i\_impact} \times 0.30) + (\text{a\_impact} \times 0.50)$ 
21: // Step 5: Asset criticality coefficient (IEC 62443)
22: AC  $\leftarrow \text{AssetCriticality}[\text{dt}]$ 
23: // Step 6: Environmental score
24: env  $\leftarrow \min(\text{T} \times \text{CIA} \times \text{AC} \times 10, 10.0)$ 
25: if kev = 1 then
26:   env  $\leftarrow \min(\text{env} \times 1.10, 10.0)$ 
27: end if
28: // Step 7: Suricata network alert context
29: if ac > 0 then
30:   sw  $\leftarrow \text{SeverityWeight}[\text{as}]$ 
31:   Sf  $\leftarrow \min((\text{ac} \times \text{sw}) / 30.0, 1.0)$ 
32: else
33:   Sf  $\leftarrow 0.0$ 
34: end if
35: // Step 8: Final score
36: score  $\leftarrow \min(\text{env} + (\text{S}_f \times 1.5), 10.0)$ 
37: score  $\leftarrow \text{Round}(\text{score}, 1)$ 
38: // Step 9: Classify risk tier
39: if score  $\geq 8.0$  then
40:   tier  $\leftarrow \text{CRITICAL}$ 
41: else if score  $\geq 6.0$  then
42:   tier  $\leftarrow \text{HIGH}$ 
43: else if score  $\geq 4.0$  then
44:   tier  $\leftarrow \text{MEDIUM}$ 
45: else
46:   tier  $\leftarrow \text{LOW}$ 
47: end if
48: return score, tier

```

Algorithm 3 UPSERT-Based Database Persistence

Input: Device list D with CVE mappings and risk scores; PostgreSQL connection parameters**Output:** All assets and vulnerabilities persisted; asset statuses updated

```

1:  $conn \leftarrow \text{CONNECT}(\text{host}, \text{port}, \text{dbname}, \text{user}, \text{password})$ 
2: for each device  $d \in D$  do
3:   SQL: INSERT INTO assets (ip, port, vendor, ...)
4:     VALUES ( $d.\text{ip}$ ,  $d.\text{port}$ ,  $d.\text{vendor}$ , ...)
5:   ON CONFLICT (ip, port) DO UPDATE SET
6:     last_seen = NOW(), status = 'ACTIVE'
7:   RETURNING id  $\rightarrow$   $\text{asset\_id}$ 
8:   for each CVE  $c \in d.\text{cves}$  do
9:     SQL: INSERT INTO vulnerabilities (asset_id, cve_id, cvss, risk_score,
    risk_tier, ...)
10:    VALUES ( $\text{asset\_id}$ ,  $c.\text{cve\_id}$ ,  $c.\text{cvss}$ , ...)
11:    ON CONFLICT (asset_id, cve_id) DO UPDATE SET
12:      risk_score = EXCLUDED.risk_score,
13:      risk_tier = EXCLUDED.risk_tier,
14:      updated_at = NOW()
15:   end for
16: end for
17: SQL: UPDATE assets SET status =
18:   CASE WHEN last_seen  $\geq$  NOW() - INTERVAL '10 minutes'
19:     THEN 'ACTIVE' ELSE 'INACTIVE' END
20:  $conn.\text{COMMIT}()$ 
21:  $conn.\text{CLOSE}()$ 

```

Figure 4.2: Algorithm 3: UPSERT-Based Database Persistence

4.2 External APIs/SDKs

CAVE-OT integrates several third-party libraries and public data APIs across its pipeline, dashboard, and database layers. The table below lists each dependency, its role in the system, and the specific functions or endpoints used.

API and Version	Description	Purpose of Usage	Endpoint / Function Used
Flask (3.x)	Python web framework for building web apps and REST APIs	Serves the dashboard and all JSON API endpoints	Flask(), @app.route(), render_template(), jsonify()
psycopg2 (2.9.x)	PostgreSQL adapter for Python	Connects to the cave_ot database and executes all queries	psycopg2.connect(), cursor.execute(), RealDictCursor
scikit-learn (1.x)	Machine learning library for Python	Trains TF-IDF vectorisers and computes cosine similarity	TfidfVectorizer, cosine_similarity()
joblib (1.x)	Python serialisation library	Saves and loads trained TF-IDF model files from disk	joblib.dump(), joblib.load()
pandas (2.x)	Data analysis and manipulation library	Loads and filters CVE datasets during training and mapping	pd.read_csv(), DataFrame.iterrows(), DataFrame.iloc[]
NVD REST API v2.0	NIST National Vulnerability Database public REST API	Source of CVE records with CVSS scores and CIA impact values	services.nvd.nist.gov/rest/json/cves/2.0
EPSS API (FIRST.org)	Exploit Prediction Scoring System API by FIRST	Retrieves daily exploit probability scores per CVE	api.first.org/data/1.0/epss
CISA KEV Catalogue	CISA Known Exploited Vulnerabilities JSON feed	Flags CVEs confirmed as actively exploited in the wild	cisa.gov/.../known_exploited_vulnerabilities.json
Chart.js (4.x)	JavaScript charting library on HTML5 canvas	Renders risk, CVE distribution, and zone comparison charts	new Chart(), type: bar, doughnut, radar
Bootstrap (5.3)	CSS and JavaScript UI component framework	Provides responsive layout and styling for the dashboard	bootstrap.min.css, bootstrap.bundle.min.js

Table 4.1: External APIs and SDKs used in CAVE-OT

4.3 Testing Details

Once the system has been successfully developed, testing has to be performed to ensure that the system is working as intended.

4.3.1 Unit Testing

Each unit test is designed to test a specific function or method independently from other components, helping to identify issues directly related to the functionality being tested.

TC ID	Test Objective	Pre-condition	Steps	Test Data	Expected and Actual Result	Post-condition	Pass Fail
TC-01	Verify exploit maturity for high EPSS	Risk scorer module loaded	Call exploit maturity function with epss 0.80	epss = 0.80	Returns 1.00	No state change	Pass
TC-02	Verify exploit maturity for low EPSS	Risk scorer module loaded	Call exploit maturity function with epss 0.05	epss = 0.05	Returns 0.91	No state change	Pass
TC-03	Verify remediation level for KEV confirmed CVE	Risk scorer module loaded	Call remediation level function with kev 1	kev = 1	Returns 1.00	No state change	Pass
TC-04	Verify remediation level when official fix exists	Risk scorer module loaded	Call remediation level function with kev 0	kev = 0	Returns 0.95	No state change	Pass

TC ID	Test Objective	Pre-condition	Steps	Test Data	Expected and Actual Result	Post-condition	Pass Fail
TC-05	Verify OT CIA reweighting with availability only	Risk scorer module loaded	Call CIA score function with only availability set	c=0.0, i=0.0, a=1.0	Returns 0.50	No state change	Pass
TC-06	Verify Suricata factor is zero with no alerts	Risk scorer module loaded	Call Suricata factor function with alert count zero	count=0, severity=1	Returns 0.0	No state change	Pass
TC-07	Verify Suricata factor is capped at 1.0	Risk scorer module loaded	Call Suricata factor function with count 30	count=30, severity=1	Returns 1.0	No state change	Pass
TC-08	Verify final risk score clamped to 10.0	Risk scorer module loaded	Call risk score function with maximum inputs on PLC	cvss=9.8, epss=0.80, kev=1, PLC	Returns 10.0	No state change	Pass
TC-09	Verify minimum score for zero CVSS CVE	Risk scorer module loaded	Call risk score function with all zero inputs	cvss=0.0, epss=0.0, kev=0	Returns 0.0	No state change	Pass
TC-10	Verify CRITICAL tier for score above 8.0	Risk scorer module loaded	Call risk tier function with score 9.0	score = 9.0	Returns CRITICAL	No state change	Pass
TC-11	Verify HIGH tier assigned for score 6.5	Risk scorer module loaded	Call risk tier function with score 6.5	score = 6.5	Returns HIGH	No state change	Pass

4. Implementation and Testing

TC ID	Test Objective	Pre-condition	Steps	Test Data	Expected and Actual Result	Post-condition	Pass Fail
TC-12	Verify OT model selected for OT zone device	Models and assets file loaded	Call CVE mapper with Siemens S7-300 in OT zone	Siemens S7-300, zone=OT, fw=V3.2.5	CVEs returned with similarity above 0.15	Device updated with CVE list	Pass
TC-13	Verify general model selected for IT zone device	Models and assets file loaded	Call CVE mapper with WinCC OA in IT zone	WinCC OA, zone=IT, fw=3.17	CVEs returned with similarity above 0.20	Device updated with CVE list	Pass
TC-14	Verify empty device query returns empty list	Models loaded	Call CVE mapper with blank vendor and product	vendor=empty, product=empty	Empty list returned, no error	No state change	Pass
TC-15	Verify firmware filter includes CVE within range	CVE database loaded	Call version filter with firmware inside range	fw=3.2, range 3.0 to 3.5	CVE included in results	Filtered list returned	Pass
TC-16	Verify firmware filter excludes CVE outside range	CVE database loaded	Call version filter with firmware outside range	fw=2.0, range 3.0 to 3.5	CVE excluded from results	Filtered list returned	Pass
TC-17	Verify database connection with valid credentials	PostgreSQL running, cave ot database exists	Call connection test with valid config	host=localhost, port=5432	Returns True and prints version	Connection closed cleanly	Pass
TC-18	Verify new asset inserted into database	Database connected, assets table exists	Call upsert asset with new IP and port	ip=127.0.0.1, port=10201	Returns integer asset ID	Row inserted in assets table	Pass

TC ID	Test Objective	Pre-condition	Steps	Test Data	Expected and Actual Result	Post-condition	Pass Fail
TC-19	Verify existing asset updated and not duplicated	Asset already exists in database	Call upsert asset with same IP and port	ip=127.0.0.1, port=10201, fw=V3.3	Updates last seen, no duplicate row	Single row remains in table	Pass
TC-20	Verify asset marked INACTIVE after 10 minutes	Asset last seen older than 10 minutes	Call asset status update function	last seen = 15 minutes ago	Status updated to INACTIVE	Status changed in database	Pass

Table 4.2: Unit Test Cases for CAVE-OT System

Chapter 5

Conclusions and Future Work

5.1 Conclusion

This project set out to address a fundamental gap in industrial cybersecurity: the inability of conventional vulnerability management tools to prioritise threats meaningfully in operational technology environments. Existing tools such as Nozomi Networks and Tenable.ot suffer from alert overload, rely on CVSS scores that ignore production impact, and provide no attack-path reasoning. These gaps exist because such tools were designed for IT-first security models, focus on severity rather than production failure, and risk OT disruption through active scanning.

CAVE-OT was designed and implemented as a direct response to these limitations. The system is an AI-assisted decision-support system designed for industrial IT/OT environments that prioritises vulnerabilities based on context, impact, and attack paths, and focuses on explanation rather than just detection. The current iteration successfully delivers the complete data acquisition and risk prioritisation pipeline. Assets are discovered passively from a device inventory, matched to known CVEs using a trained TF-IDF cosine similarity model, and scored using a contextual risk formula that integrates CVSS v3.1 temporal scoring, EPSS exploit probability, CISA KEV confirmation status, OT-specific CIA reweighting aligned with NIST SP 800-82, and asset criticality coefficients derived from IEC 62443. Results are persisted to a PostgreSQL database using UPSERT-based logic that supports continuous re-scanning without data duplication, and a Flask-based monitoring dashboard provides real-time visibility into asset states, vulnerability distributions, and risk tiers.

The core motivation for this contextual approach is well-founded: only approximately 17% of critical CVSS vulnerabilities are ever exploited in real attacks, and CVSS scores are static while OT threats are dynamic. A SCADA server vulnerability with a lower CVSS score of 7.0 carries higher overall risk than an Office PC vulnerability scored at

9.5, because it directly impacts production. CAVE-OT captures this distinction through its OT CIA reweighting, which assigns a 50% weight to availability, 30% to integrity, and 20% to confidentiality, inverting the IT-centric defaults of standard CVSS scoring.

A direct comparison with existing tools confirms the differentiation of CAVE-OT: neither Nozomi Networks nor Tenable.ot provide attack path visibility, downtime awareness, decision explanation, or failure impact analysis, whereas CAVE-OT delivers all five criteria with strong OT-aware industrial context awareness.

The unit test suite of 20 test cases, all passing, validates the correctness of every core function in the risk scoring engine, CVE mapping pipeline, and database persistence layer. The system is deployed and operational against a simulated water treatment facility asset inventory comprising six heterogeneous OT devices spanning DNP3, S7comm, Modbus, BACnet, and HTTP protocols.

5.2 Future Work

While the current iteration establishes a solid and validated foundation, several significant capabilities remain planned for subsequent development phases, as outlined in the project roadmap.

Attack Path Correlation Engine

The high-level system overview identifies Attack Path Correlation as a policy-constrained component responsible for IT-to-OT attack mapping, lateral movement analysis, and exploitable path discovery. The next development phase will implement this engine using NetworkX directed graphs, where nodes represent network assets and edges represent exploitable vulnerability relationships. The engine will identify multi-hop attack paths from IT entry points through the DMZ to critical OT devices such as PLCs and SCADA servers, enabling operators to visualise how a compromised enterprise workstation could propagate to a safety-critical process controller.

LLM-Based Remediation Advisor

The architecture includes an LLM-Based Remediation Advisor providing OT-safe remediation guidance, risk-prioritised actions, and recommendations with minimal operational impact. This component will be implemented using a Retrieval-Augmented Generation (RAG) pipeline, where a vector index of policy documents, CVE advisories, and remediation templates is queried at inference time to ground the language model's output in

verified industrial guidance. The Human-in-the-Loop interface will allow SOC analysts and OT engineers to approve or reject recommendations before any action is recorded.

Neo4j Graph Database Integration

The current PostgreSQL implementation will be extended with a Neo4j graph database to persist network topology nodes and attack path edges natively as a property graph. This will enable efficient traversal queries for path discovery and visualisation of lateral movement chains directly within the dashboard.

Bibliography

- [1] Nozomi Networks, “OT/IoT Vulnerability Management Platform,” 2024. [Online]. Available: nozominetworks.com
- [2] Australian Cyber Security Magazine, “The Economic Impact of ICS Vulnerabilities,” 2023. [Online]. Available: australiancybersecuritymagazine.com.au
- [3] Kaspersky, “Why CVSS Alone Is Not Enough for Risk-Based Vulnerability Management,” 2023. [Online]. Available: kaspersky.com
- [4] Nexus Connect, “CVSS Scores Are No Longer Enough: The Move to Context-Driven Vulnerability Management,” 2024. [Online]. Available: nexusconnect.io
- [5] FIRST Organization, “Common Vulnerability Scoring System (CVSS) v3.1 Specification Document,” 2019. [Online]. Available: first.org/cvss
- [6] FIRST Organization, “Exploit Prediction Scoring System (EPSS),” 2023. [Online]. Available: first.org/epss
- [7] National Institute of Standards and Technology, “National Vulnerability Database (NVD),” 2024. [Online]. Available: nvd.nist.gov
- [8] Cybersecurity and Infrastructure Security Agency, “Known Exploited Vulnerabilities Catalogue,” 2024. [Online]. Available: cisa.gov
- [9] National Institute of Standards and Technology, “Guide to Industrial Control Systems (ICS) Security: NIST Special Publication 800-82 Revision 2,” Tech. Rep. SP 800-82 Rev. 2, Gaithersburg, MD, 2015. [Online]. Available: doi.org/10.6028/NIST.SP.800-82r2
- [10] International Electrotechnical Commission, “IEC 62443: Security for Industrial Automation and Control Systems,” International Standard IEC 62443, 2018.
- [11] Tenable, “Tenable OT Security,” 2024. [Online]. Available: tenable.com

- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [13] G. Salton and C. Buckley, “Term-Weighting Approaches in Automatic Text Retrieval,” *Information Processing and Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [14] Pallets Projects, “Flask: The Python Micro Framework for Building Web Applications,” 2024. [Online]. Available: flask.palletsprojects.com
- [15] The PostgreSQL Global Development Group, “PostgreSQL: The World’s Most Advanced Open Source Relational Database,” 2024. [Online]. Available: postgresql.org
- [16] Chart.js Contributors, “Chart.js: Simple Yet Flexible JavaScript Charting,” 2024. [Online]. Available: chartjs.org
- [17] Bootstrap Team, “Bootstrap: The Most Popular HTML, CSS, and JS Library,” 2024. [Online]. Available: getbootstrap.com